

Écrire un programme

Une méthode pour passer d'une consigne à un programme, dans Thonny

Nicholas Wolff · CC BY-SA 4.0

Sommaire

1

Exemple

Le programme du cercle, écrit pas à pas dans Thonny : l'enregistrer, l'exécuter, le compléter

2

Les erreurs

Erreurs de syntaxe, de type et de calcul

3

Résumé

La méthode, étape par étape

Tâche qu'on essaye d'automatiser (programmer)

Écrire un programme qui demande le rayon d'un cercle à la personne qui l'utilise, pour ensuite calculer et afficher :

- son diamètre ($2r$)
- sa circonférence ($2\pi r$)
- sa surface (πr^2)

À chaque fois, le programme ne doit pas juste afficher le résultat, mais préciser ce qui est en train d'être affiché, par exemple : `Le diamètre du cercle est 23.4`

Comprendre la tâche

Bien lire la consigne du début à la fin.

On sait qu'on a compris la consigne si on est capable de faire soi-même ce que le programme devra faire, sans le programmer.

Ici, on peut vérifier avec un exemple de rayon qu'on invente.

On décide de jouer le rôle du programme pour un rayon de 10, en faisant les calculs de tête :

```
rayon du cercle ? 10
Le diamètre est 20.0
La circonférence est 62.831
La surface est 314.15
```

Écrire un programme qui demande le rayon d'un cercle à la personne qui l'utilise, pour ensuite calculer et afficher :

- son diamètre ($2r$)
- sa circonférence ($2\pi r$)
- sa surface (πr^2)

À chaque fois, le programme ne doit pas juste afficher le résultat, mais préciser ce qui est en train d'être affiché, par exemple : `Le diamètre du cercle est 23.4`

Se faire une idée de la forme du programme

Pour un programme simple, lire la consigne suffit à se faire une idée de l'ordre des choses qui vont se passer :

- Demander et stocker le rayon
- Calculer le diamètre
- Afficher le diamètre

… encore des calculs et des affichages

Écrire un programme qui demande le rayon d'un cercle à la personne qui l'utilise, pour ensuite calculer et afficher :

- son diamètre ($2r$)
- sa circonférence ($2\pi r$)
- sa surface (πr^2)

À chaque fois, le programme ne doit pas juste afficher le résultat, mais préciser ce qui est en train d'être affiché, par exemple : `Le diamètre du cercle est 23.4`

Identifier les variables, les nommer, choisir leur type

Activité · 2 min

On cherche les variables du programme : chaque information qu'il manipule en sera une.

- Quelles informations ce programme manipule-t-il ?
- Quel nom donner à chacune ?
- Quel type pour chacune ?

Écrire un programme qui demande le rayon d'un cercle à la personne qui l'utilise, pour ensuite calculer et afficher :

- son diamètre ($2r$)
- sa circonférence ($2\pi r$)
- sa surface (πr^2)

À chaque fois, le programme ne doit pas juste afficher le résultat, mais préciser ce qui est en train d'être affiché, par exemple : `Le diamètre du cercle est 23.4`

Identifier les variables, les nommer, choisir leur type

Les informations qu'on manipule dans ce programme sont :
le rayon, le diamètre, la circonférence, la surface.

Ce qui donne les **noms** des variables :
`rayon`, `diamètre`, `circonférence`, `surface`

Types des variables :

- Pour `rayon`, on a le choix : soit `int`, soit `float`. Si on choisissait `int`, on ne pourrait pas parler d'un cercle de rayon `2.5` : `float` est donc un meilleur choix.
- `diamètre`, `circonférence` et `surface` sont calculés à partir de `rayon` : `float`

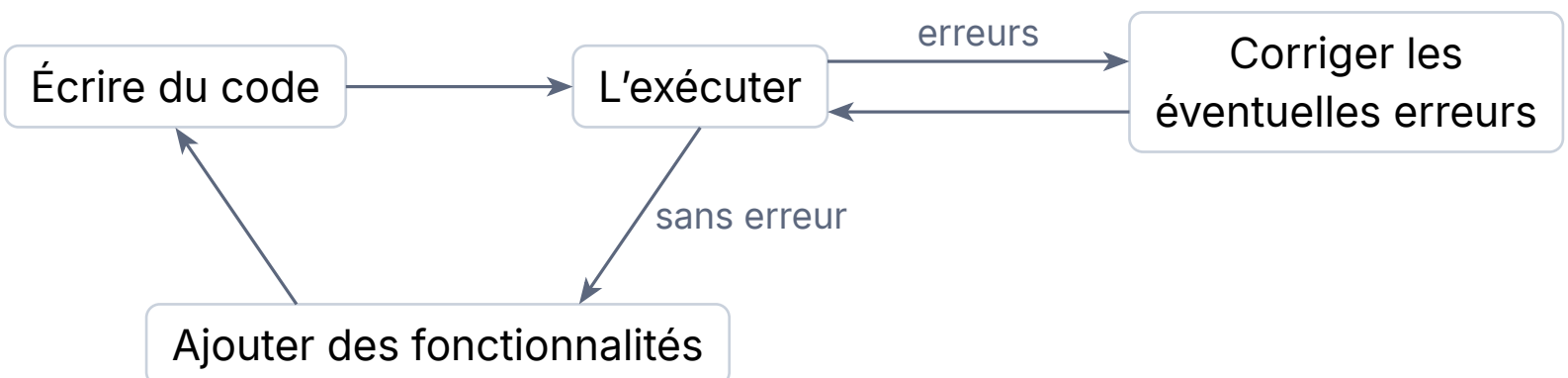
Écrire un programme qui demande le rayon d'un cercle à la personne qui l'utilise, pour ensuite calculer et afficher :

- son diamètre ($2r$)
- sa circonférence ($2\pi r$)
- sa surface (πr^2)

À chaque fois, le programme ne doit pas juste afficher le résultat, mais préciser ce qui est en train d'être affiché, par exemple : `Le diamètre du cercle est 23.4`

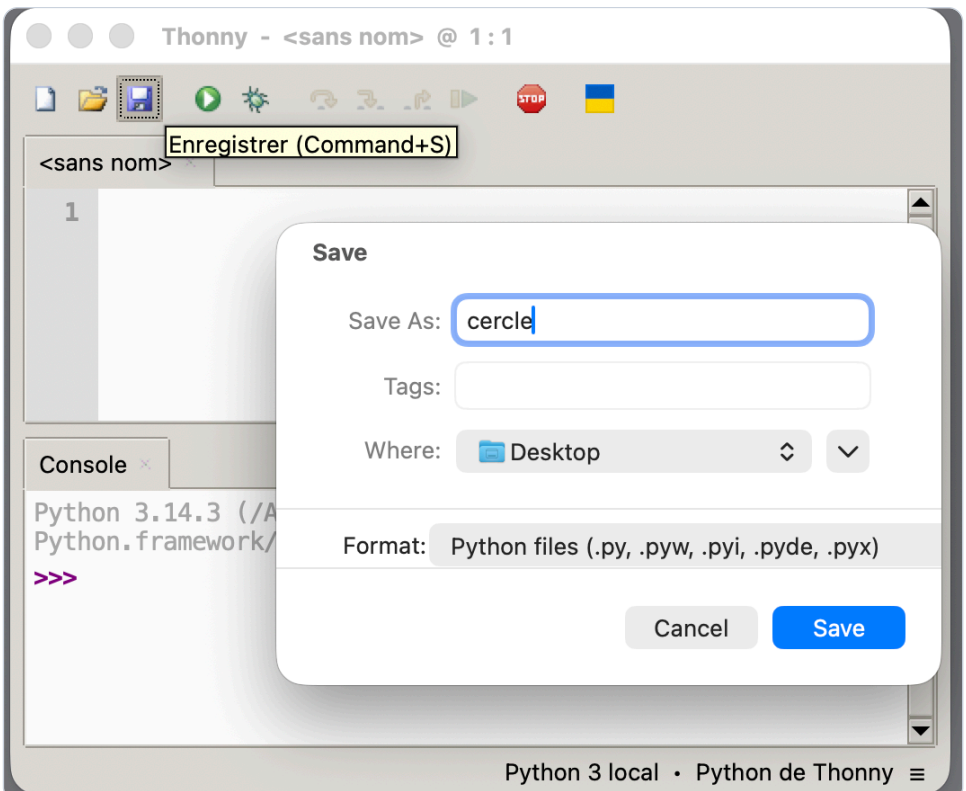
Commencer à écrire le programme

On sait maintenant ce que le programme doit contenir. On va l'écrire petit à petit :



Les différentes sortes d'erreurs, et comment les trouver, sont l'objet de la partie « Les erreurs ».

Enregistrer le programme dès le début



Avant d'écrire la première ligne, on enregistre le programme dans un fichier :

- on lui trouve un nom, par exemple `cercle.py` ou `calculs-cercle.py` ;
- on l'enregistre sur le Bureau, que Thonny appelle `Desktop` : comme ça, on voit immédiatement le fichier apparaître.

On enregistre avec ⌘ + S ou la disquette de la barre d'outils. Une * à côté du nom de l'onglet indique des modifications pas encore enregistrées.

Ensuite, chaque fois qu'on exécute le programme, Thonny l'enregistre automatiquement.

Demander et stocker le rayon

Demander un nombre et le stocker dans une variable `rayon` de type `float`

cercle.py

```
1  rayon = float(input("rayon du cercle ? "))
2
```

Console

```
  rayon du cercle ? 22
>>>
```

On exécute ce bout de programme : la question s'affiche dans la console, on tape un nombre, et le programme se termine sans erreur.

Écrire un programme qui demande le rayon d'un cercle à la personne qui l'utilise, pour ensuite calculer et afficher :

- son diamètre ($2r$)
- sa circonférence ($2\pi r$)
- sa surface (πr^2)

À chaque fois, le programme ne doit pas juste afficher le résultat, mais préciser ce qui est en train d'être affiché, par exemple : `Le diamètre du cercle est 23.4`

Calculer le diamètre

Variable `diamètre` de type `float`

cercle.py

```
1  rayon = float(input("rayon du cercle ? "))
2
3  diamètre = 2 * rayon
```

Console

```
>>> %Run cercle.py
  rayon du cercle ? 11
>>>
```

Pas d'erreur, mais rien de plus ne s'affiche : le diamètre est dans la variable `diamètre`, et le programme ne l'affiche pas encore.

Écrire un programme qui demande le rayon d'un cercle à la personne qui l'utilise, pour ensuite calculer et afficher :

- son diamètre ($2r$)
- sa circonférence ($2\pi r$)
- sa surface (πr^2)

À chaque fois, le programme ne doit pas juste afficher le résultat, mais préciser ce qui est en train d'être affiché, par exemple : `Le diamètre du cercle est 23.4`

Afficher le diamètre

cercle.py

```
1  rayon = float(input("rayon du cercle ? "))
2
3  diamètre = 2 * rayon
4  print("Le diamètre est", diamètre)
```

Console

```
>>> %Run cercle.py

  rayon du cercle ? 11
  Le diamètre est 22.0

>>>
```

Le programme dit ce qu'il affiche, comme le demande la consigne : `Le diamètre est 22.0`, pas juste `22.0`.
Et on vérifie le calcul de tête : $11 \times 2 = 22$.

Écrire un programme qui demande le rayon d'un cercle à la personne qui l'utilise, pour ensuite calculer et afficher :

- son diamètre ($2r$)
- sa circonférence ($2\pi r$)
- sa surface (πr^2)

À chaque fois, le programme ne doit pas juste afficher le résultat, mais préciser ce qui est en train d'être affiché, par exemple : `Le diamètre du cercle est 23.4`

Calculer et afficher la circonférence

Variable `circonférence` de type `float`

On ajoute deux lignes et on exécute.

cercle.py

```
1  rayon = float(input("rayon du cercle ? "))
2
3  diamètre = 2 * rayon
4  print("Le diamètre est", diamètre)
5
6  circonférence = 2 * 3.14159265359 * rayon
7  print("La circonférence est", circonférence)
```

Console

```
>>> %Run cercle.py

  rayon du cercle ? 1
  Le diamètre est 2.0
  La circonférence est 6.28318530718

>>>
```

On vérifie le résultat : avec un rayon de 1, la circonférence vaut 2π.

Écrire un programme qui demande le rayon d'un cercle à la personne qui l'utilise, pour ensuite calculer et afficher :

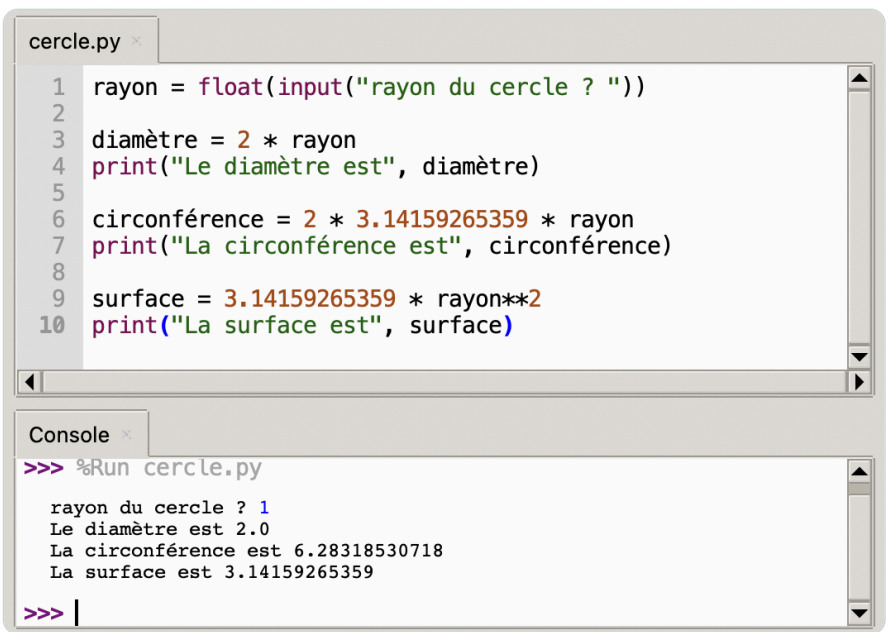
- son diamètre ($2r$)
- sa circonférence ($2\pi r$)
- sa surface (πr^2)

À chaque fois, le programme ne doit pas juste afficher le résultat, mais préciser ce qui est en train d'être affiché, par exemple : `Le diamètre du cercle est 23.4`

Calculer et afficher la surface

Variable `surface` de type `float`

On ajoute deux lignes et on exécute.



On vérifie le résultat : avec un rayon de 1, la surface vaut π.

Écrire un programme qui demande le rayon d'un cercle à la personne qui l'utilise, pour ensuite calculer et afficher :

- son diamètre ($2r$)
- sa circonférence ($2\pi r$)
- sa surface (πr^2)

À chaque fois, le programme ne doit pas juste afficher le résultat, mais préciser ce qui est en train d'être affiché, par exemple : `Le diamètre du cercle est 23.4`

Amélioration qu'on découvre en programmant

cercle.py

```
1  pi = 3.14159265359
2
3  rayon = float(input("rayon du cercle ? "))
4
5  diamètre = 2 * rayon
6  print("Le diamètre est", diamètre)
7
8  circonférence = 2 * pi * rayon
9  print("La circonférence est", circonférence)
10
11 surface = pi * rayon**2
12 print("La surface est", surface)
```

Console

```
>>> %Run cercle.py

  rayon du cercle ? 1
  Le diamètre est 2.0
  La circonférence est 6.28318530718
  La surface est 3.14159265359

>>>
```

La valeur `3.14159265359` revient plusieurs fois dans le programme. Pour éviter le risque d'erreur en l'écrivant, on la stocke dans une variable `pi`. On utilise cette variable aux deux endroits où on en a besoin.

On exécute à nouveau le programme, pour vérifier qu'il donne toujours les mêmes résultats.

Deux programmes qui font ce qui est demandé

Ces deux programmes sont corrects. Il y a toujours beaucoup de manières d'écrire un programme.

Une version très explicite :

```
pi = 3.14159265359

rayon = float(input("rayon du cercle ? "))

diamètre = 2 * rayon
print("Le diamètre est", diamètre)

circonférence = 2 * pi * rayon
print("La circonférence est", circonférence)

surface = pi * rayon**2
print("La surface est", surface)
```

Une version compacte :

```
pi = 3.14159265359
r = float(input("rayon du cercle ? "))
print("Le diamètre est", r * 2)
print("La circonférence est", 2 * pi * r)
print("La surface est", pi * r**2)
```

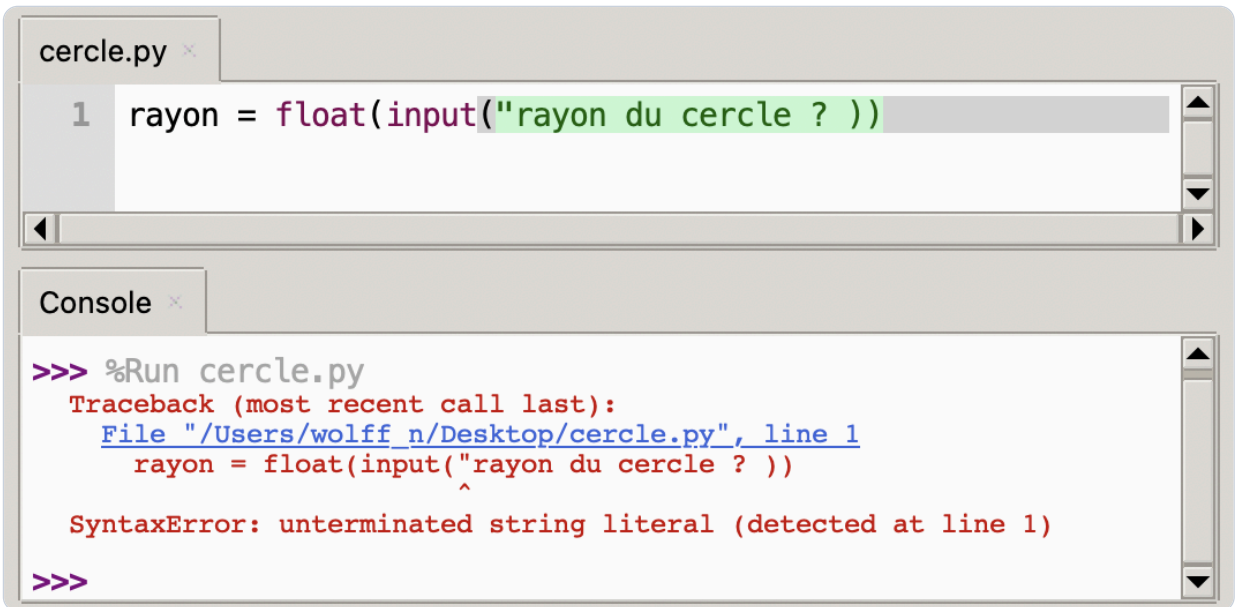
Sommaire

1
Exemple
Le programme du cercle, écrit pas à pas dans Thonny : l'enregistrer, l'exécuter, le compléter

2
Les erreurs
Erreurs de syntaxe, de type et de calcul

3
Résumé
La méthode, étape par étape

Exemple d'erreur de syntaxe



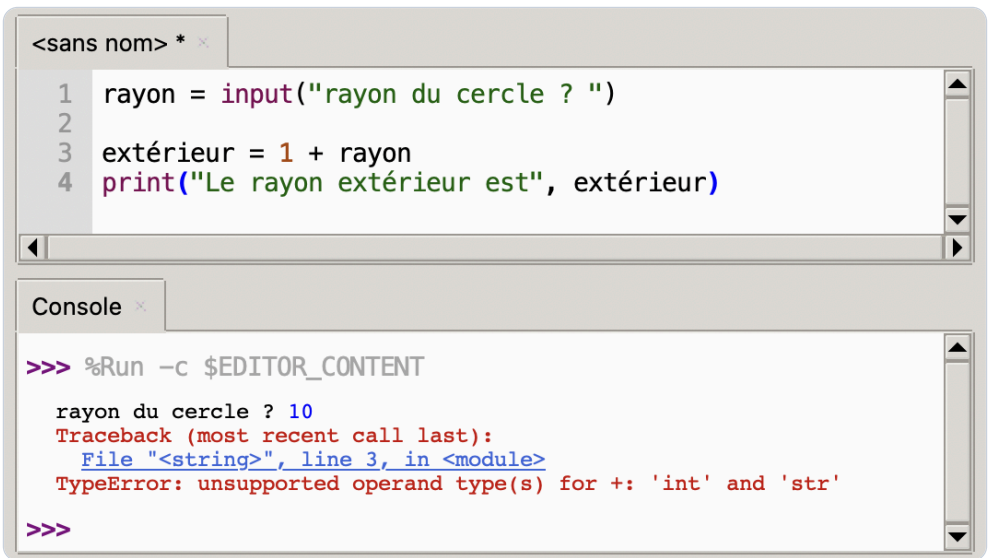
Ici, on a une erreur de syntaxe : le programme ne respecte pas les règles d'écriture de Python, qui ne peut pas l'exécuter.

- On peut la voir dans l'éditeur en haut : le bloc vert indique que la chaîne de caractères ne se termine jamais.
- Dans la console en bas, on voit un message d'erreur, en anglais, avec un lien qui permet d'arriver sur la ligne où se trouve l'erreur.

L'erreur : il manque le " qui ferme la chaîne `"rayon du cercle ?` . C'est ce que dit la console : *unterminated string literal*, une chaîne de caractères qui ne se termine pas.

Exemple d'erreur de type

Admettons que le programme qu'on nous demande d'écrire soit légèrement différent : on doit calculer le rayon extérieur d'une roue étant donné le rayon intérieur, en ajoutant 1 cm.



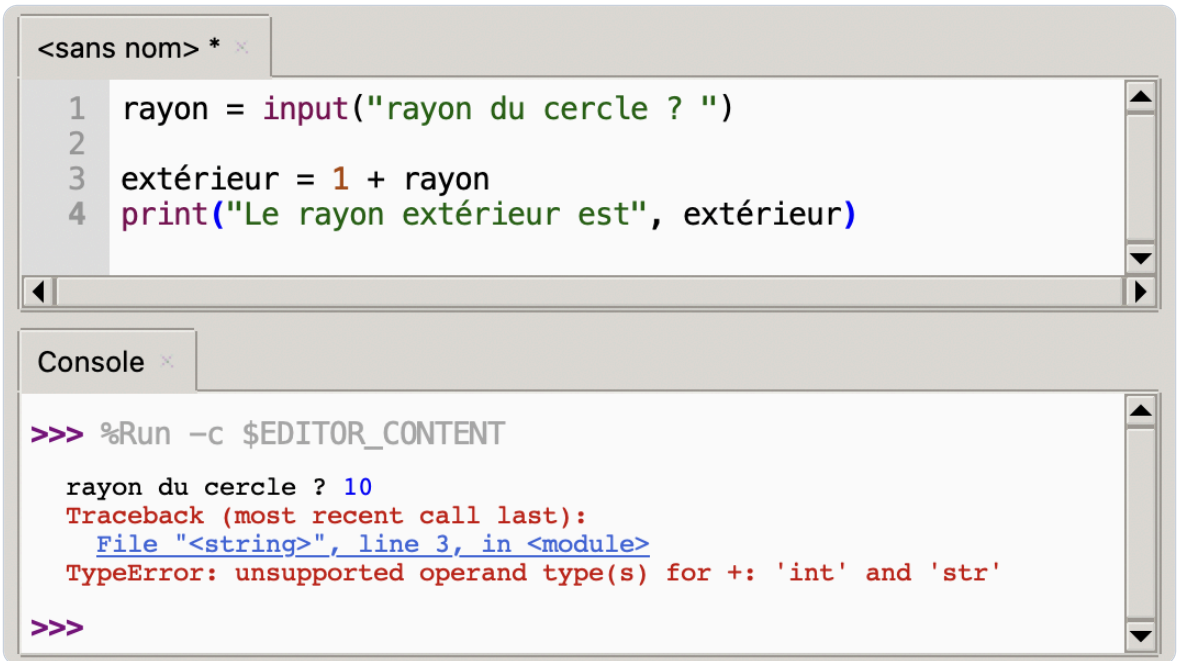
Dans la console en bas, on voit un message d'erreur :

- qui nous dit qu'il y a une erreur de type (on ne peut pas additionner un `int` et un `str`) ;
- qui nous indique la ligne 3.

Activité · 3 min

Chercher comment corriger le programme.

Exemple d'erreur de type

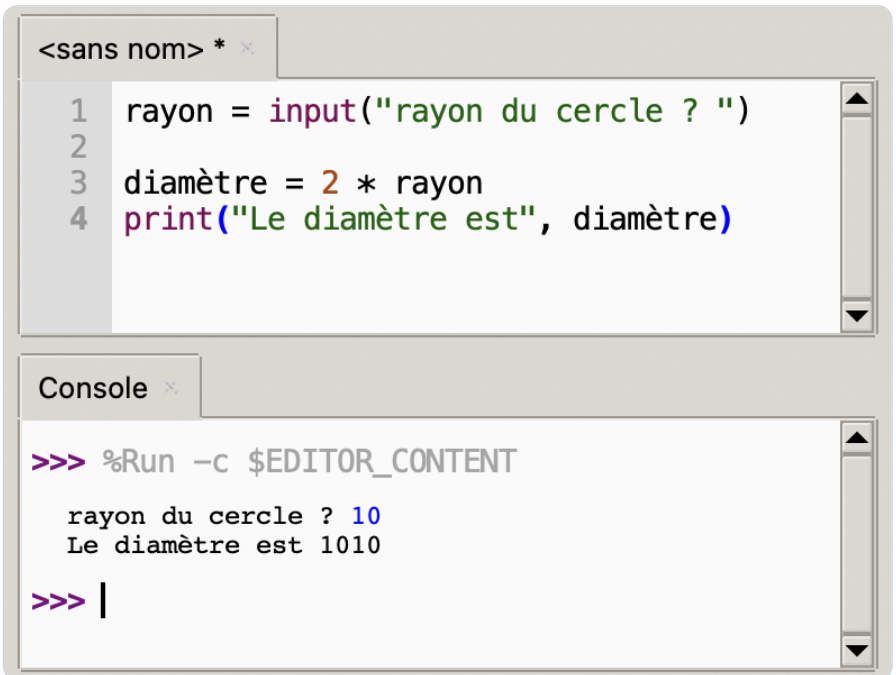


En réalité, **l'erreur est avant la ligne 3**. On a oublié de convertir vers `float` la réponse donnée par la personne qui utilise le programme. Voici le programme corrigé :

```
rayon = float(input("rayon du cercle ? "))

extérieur = 1 + rayon
print("Le rayon extérieur est", extérieur)
```

Une erreur de type, mais sans message d'erreur



Ici, le programme ne contient pas d'erreur de syntaxe : il s'exécute.

Et pendant l'exécution, aucun message d'erreur ne s'affiche non plus : il va jusqu'au bout. Thonny ne va donc pas nous aider à trouver l'erreur.

Il faut essayer de comprendre ce que l'on voit : on a répondu 10 pour le rayon, et on a obtenu 1010.

C'est parce qu'on a oublié de *convertir* la réponse de la personne qui utilise le programme vers `float`.

On découvre qu'on peut « multiplier » un `int` et un `str` : ça répète le `str` un certain nombre de fois :

```
3 * "tsk"  →  "tsktsktsk"
```

Exemple d'erreur de calcul

On a écrit une autre version du programme du cercle. Elle s'exécute sans message d'erreur, mais ne donne pas toujours les bons résultats. Pour l'instant, on ne voit pas son code : on l'exécute pour tester.

Test avec un rayon de 0 :

```
rayon du cercle ? 0
Le diamètre est 0.0
La circonférence est 0.0
La surface est 0.0
```

Qu'est-ce que ce test nous apprend ?

Écrire un programme qui demande le rayon d'un cercle à la personne qui l'utilise, pour ensuite calculer et afficher :

- son diamètre ($2r$)
- sa circonférence ($2\pi r$)
- sa surface (πr^2)

À chaque fois, le programme ne doit pas juste afficher le résultat, mais préciser ce qui est en train d'être affiché, par exemple : `Le diamètre du cercle est 23.4`

Exemple d'erreur de calcul

Le test avec 0 ne nous a presque rien appris. Essayons avec une autre valeur qui nous apprendra peut-être plus de choses.

Test avec un rayon de 1 :

```
rayon du cercle ? 1
Le diamètre est 2.0
La circonférence est 6.28318530718
La surface est 3.14159265359
```

Qu'est-ce que ce test nous apprend ?

Écrire un programme qui demande le rayon d'un cercle à la personne qui l'utilise, pour ensuite calculer et afficher :

- son diamètre ($2r$)
- sa circonférence ($2\pi r$)
- sa surface (πr^2)

À chaque fois, le programme ne doit pas juste afficher le résultat, mais préciser ce qui est en train d'être affiché, par exemple : `Le diamètre du cercle est 23.4`

Exemple d'erreur de calcul

Avec 1, on a pu voir qu'on a bien multiplié par 2, par 2π et par π. Pas d'erreur évidente. Pourtant il y a bien une erreur dans ce programme. Essayons de trouver une valeur qui va la révéler.

Test avec un rayon de 10 :

```
rayon du cercle ? 10
Le diamètre est 20.0
La circonférence est 62.8318530718
La surface est 31.4159265359
```

De bonnes valeurs de test :

- permettent de découvrir les erreurs ;
- peuvent être vérifiées à la main.

```
pi = 3.14159265359

rayon = float(input("rayon du cercle ? "))

diamètre = 2 * rayon
print("Le diamètre est", diamètre)

circonférence = 2 * pi * rayon
print("La circonférence est", circonférence)

surface = pi * rayon
print("La surface est", surface)
```

Sommaire

1
Exemple
Le programme du cercle, écrit pas à pas dans Thonny : l'enregistrer, l'exécuter, le compléter

2
Les erreurs
Erreurs de syntaxe, de type et de calcul

3
Résumé
La méthode, étape par étape

Méthode

- Bien lire la consigne
- Essayer de faire soi-même ce que le programme devra faire
- Se faire une idée de la forme du programme
- Identifier les variables nécessaires, les nommer, choisir leur type
- Enregistrer le programme avant d'écrire la première ligne (trouver un bon nom)
- Écrire le début du programme, corriger les erreurs de syntaxe
- Exécuter ce petit bout de programme, corriger les éventuelles erreurs avant de continuer à avancer
- Répéter jusqu'à ce que toute la consigne soit programmée
- Inventer des valeurs de test qui permettent de vérifier le fonctionnement complet du programme. De bonnes valeurs de test :
  - permettent de découvrir les erreurs ;
  - peuvent être vérifiées à la main.

Sources

Contenu

Python — Fonctions natives : `input()`, `float()`, `print()`
docs.python.org

Python — Erreurs et exceptions
docs.python.org

Images

U.S. Army – Betty Jennings et Frances Bilas programment l'ENIAC (1946, domaine public)
commons.wikimedia.org

Thonny, l'éditeur Python des captures d'écran
thonny.org